

Assignment 4 – TensorFlow Models for Predicting Loan Default and Loss

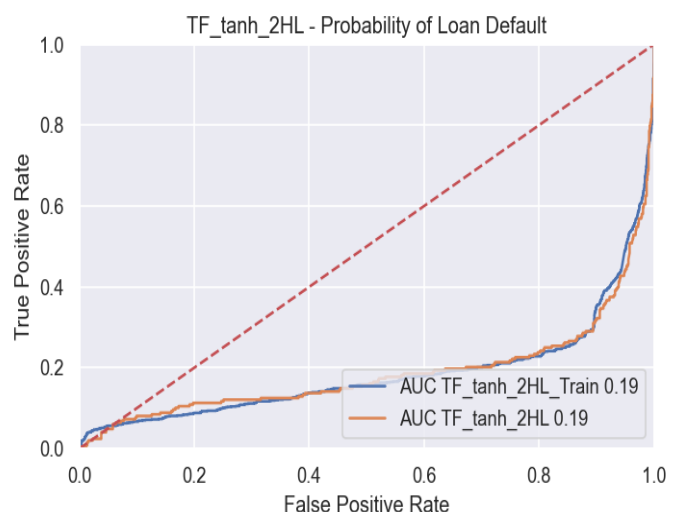
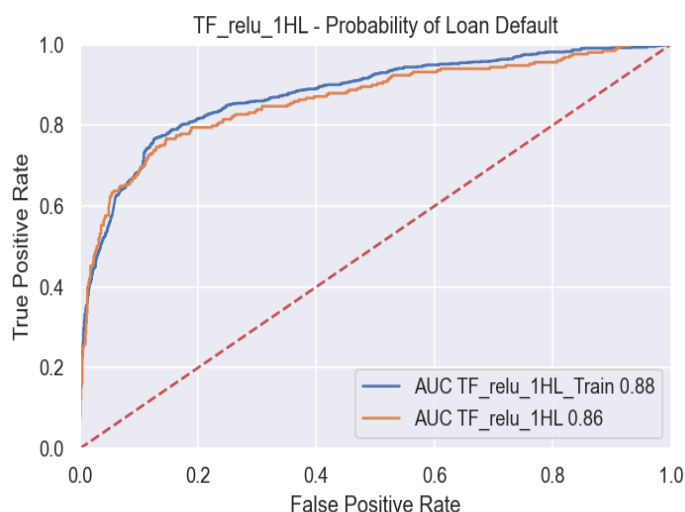
Data Preparation and Model Setup

In this analysis, the dataset was split into training (4,768 rows) and test (1,192 rows) sets at an 80/20 ratio. However, for loss amount prediction, only defaulted accounts were used, since they are the only ones with loss amounts, resulting in an 80/20 split of 941 training rows and 248 testing rows for the loss amount models.

In addition to the previously developed regression models, tree-based models such as Decision Tree, Random Forest, and Gradient Boosting, the models used and compared were: logistic regression and classification models to estimate the probability of loan default (TARGET_BAD_FLAG), and linear regression and regression models to estimate the loss amount (TARGET_LOSS_AMT) given that a default occurs. TensorFlow neural network models were added and compared with the existing models. Multiple neural network configurations were tested in order to explore how model structure affects performance. The architectures tested included three activation functions (ReLU, tanh, and sigmoid), models with one and two hidden layers, and a model with a dropout layer at a 20% dropout rate to help reduce overfitting. A variable selection technique was also applied by using the variables identified as important by the Gradient Boosting model as inputs to the neural networks. Model performance for default prediction was evaluated using classification accuracy and ROC curves, while loss prediction models were evaluated using Root Mean Squared Error (RMSE).

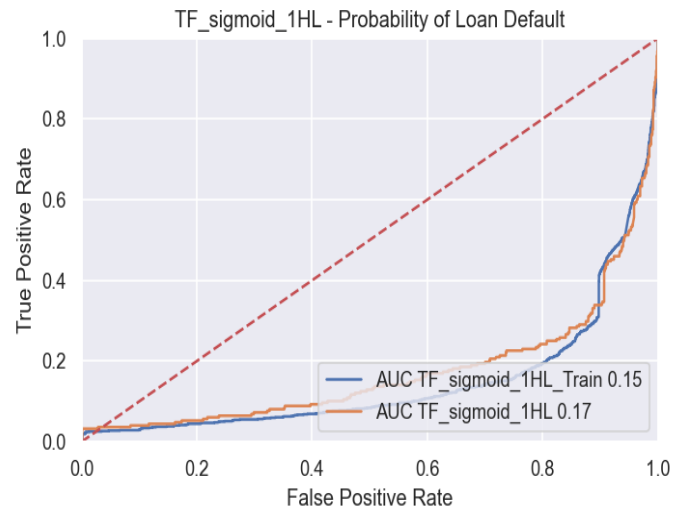
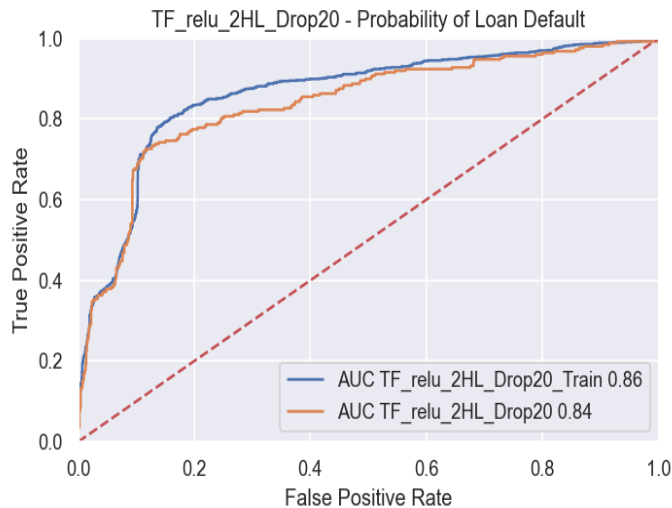
Default Probability Model Results

From the tested models that were used to predict defaulted loans, Random Forest achieved the best overall performance, with a test accuracy of 0.9094 and the highest Area Under the ROC Curve (AUC) of approximately 0.97. This indicates that the Random Forest model was very effective at distinguishing between defaulted and non-defaulted loans. The TensorFlow models appear slightly below these ensemble models, indicating somewhat lower predictive power. All TensorFlow models were trained using the variables selected through the Gradient Boosting variable importance method, which served as the variable selection technique for the neural network models.

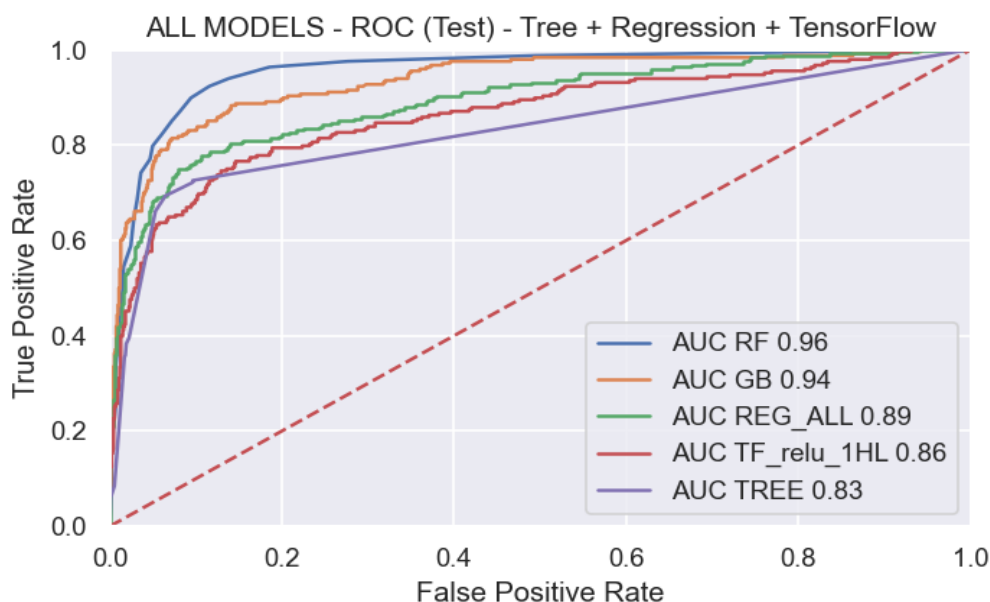


Samin Bahizad

Out of the four variations, the best-performing was TF_relu_1HL, which used a ReLU activation function with one hidden layer. This model achieved a training accuracy of 0.8674 and a test accuracy of 0.8725, with an AUC of approximately 0.86. The TF_tanh_2HL model, which used the tanh activation function with two hidden layers, achieved training accuracy of 0.8026 and test accuracy of 0.7903, lower than the ReLU model.



The TF_relu_2HL_Drop20, which used ReLU activation with two hidden layers and a 20% dropout layer which is method used to lower the risk of overfitting by randomly removing neurons during training, but this didn't make the model perform better on this dataset, as it achieved training accuracy of 0.7888 and test accuracy of 0.7777, which were slightly worse than those of the simpler ReLU model. The final model tested was TF_sigmoid_1HL, which used a sigmoid activation function with a single hidden layer. This model performed extremely poorly, achieving training and test accuracies of 0.1971 and 0.2081, respectively. Overall, the results indicate that simpler neural network structures using ReLU activation perform better than deeper or sigmoid-based architectures for this dataset. However, even the best TensorFlow model still performed slightly worse than the ensemble tree-based models like as Random Forest and Gradient Boosting.



The ROC curve clearly shows that Random Forest and Gradient Boosting dominate the other models, maintaining a higher true positive rate across most false positive rates. The TensorFlow models appear slightly below these ensemble models, indicating somewhat lower predictive power.

Loss Amount Prediction Results

The TensorFlow models also struggled to predict the loss amount for defaulted loans. Even after log-transforming the data to help handle the skewed dollar amounts, the best neural network (TF_relu_1HL_LOSS) still had a test error of \$7,580.

Other setups, such as the sigmoid and deeper tanh models, performed even worse, with errors skyrocketing to \$17,600. It seems like with only 941 training observations, the neural networks just didn't have enough data to beat the ensemble tree models at predicting precise financial losses. Gradient Boosting had the lowest test error of about \$2,620, beating Random Forest (\$3,281) and Linear Regression (\$3,493). See the chart on the right which clearly shows that Gradient Boosting provides the lowest prediction error, while the TensorFlow model has the highest RMSE among the models tested.



Conclusion

The biggest takeaway is that ensemble tree models (Random Forest and Gradient Boosting) are the clear champions for this type of dataset. Both pushed past 90% test accuracy, which is impressive. It seems that they are great at picking up on those "if-this-then-that" relationships in financial data, such as how a high debt-to-income ratio combined with a recent delinquency is a higher risk. In comparison, the TensorFlow neural networks were a bit of a struggle. The simplest one (ReLU with one layer) performed better than when we added complexity. The reason could be that, with only about 941 data rows for loss amounts, the neural networks didn't have enough "examples" to learn from, whereas the tree models were much more efficient with the limited data.

Upon further analysis, the Gradient Boosting Model remains the top model I would recommend and deploy to production, as it has the best overall performance.